3DSceneBench: Can Coding Agents Build Functional and Versatile 3D Worlds?

Yani Zhang[1*] Xinyu Wang[1*] Hanwei Wu[1*] Jindu Wang[1] Siyuan Luo[1]
Jason Pho[1] Zonglin Di[2] Yiyou Sun[2] Yi Zhu[1†]

[1]Prana Labs [2]University of California, Berkeley

[*]Equal Contribution [†]Corresponding Author

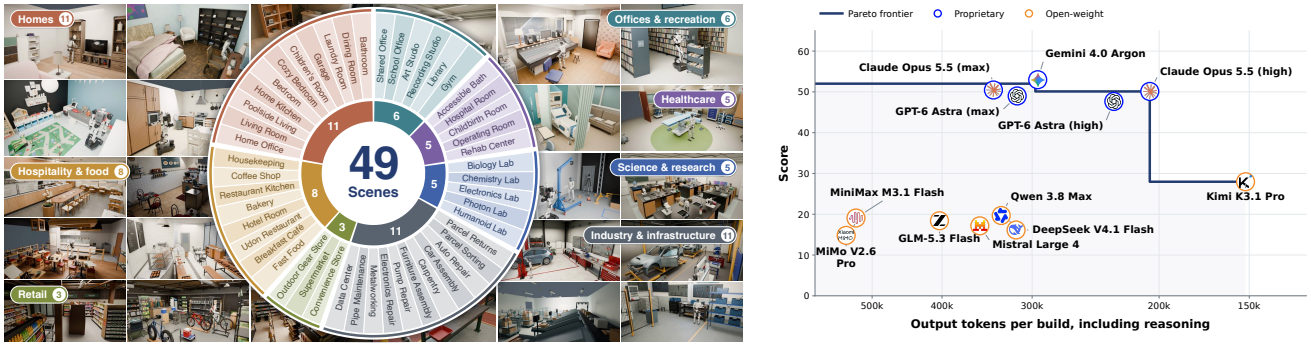


Figure 1. **3DSceneBench evaluates whether coding agents can build one functional 3D world that supports all the activities of a room.** Left: 49 whole-room scenes across seven categories, with 14,219 objects and 471 public tasks. Right: the score (mean of geometry and functionality) against output tokens per build, for ten models in 12 configurations. Blue marks proprietary models and orange marks open-weight models. The token axis decreases to the right, so the line marks the Pareto frontier of a higher score at lower token use.

Abstract

Can coding agents build one simulated world that supports a scene's many uses? Existing benchmarks check how a built scene looks, or whether it reproduces one observed event, not whether it works as a whole. We introduce 3DSceneBench, which evaluates the built world itself. Hidden tests act on one submitted world, from single physical interactions to multistage tasks, and compare each outcome with the one in a reference world, so a world scores well only when it serves all the uses of its room. The benchmark covers 49 whole-room scenes in seven categories, with 14,219 objects, 471 public tasks, and 36,871 tests, and reports appearance, geometry, and functionality. Across ten models, the best worlds reach 79.3 on appearance and 62.1 on geometry, but only 42.1 on functionality, and the leading agents complete about a quarter of their tasks in full. Most of what they lose falls on objects they never build, and the rest often work in one way and fail in another. Progress should therefore be measured by how reliably one world supports its activities without being rebuilt.

1. Introduction

For decades, people have wanted to recreate the world we live in inside a simulation, where digital beings perceive, explore, and act, and where intelligence can grow through experience as ours does. Interactive simulated environments bring this goal closer. They have become a vital ground for embodied intelligence, where agents learn from everyday activities and are evaluated under controlled and repeatable conditions [23, 33].

Recent advances in multimodal coding agents open a practical route to building such environments automatically. Coding agents assemble scene assets, specify physical properties, and refine spatial arrangements, using rendering and simulation to inspect and revise what they have built [19, 38, 44]. World construction thus becomes an iterative and programmable process, which raises a question: *what makes a construction successful?* Existing benchmarks answer this in part. Visual and geometric correspondence establishes what a scene contains and how it is arranged [12, 16, 63], and reproducing an observed or described event gives evidence about its dynamics [41, 42, 48, 59]. Each target covers one aspect, but whether the scene works across its objects, mechanisms, and activities remains open.

A real kitchen was never built for a single task. Its cabinets, appliances, containers, and work surfaces serve many activities, and each activity relies on a different combination of them. A world that opens one cabinet correctly can leave the rest of the kitchen unusable, and a world that supports one activity may not support the next. Two properties therefore matter. *Functionality* asks whether the objects, mechanisms, and physical relationships of a scene work as intended, alone and together. *Versatility* asks whether one constructed world supports different activities without being rebuilt. What is missing is a systematic examination of the constructed world itself rather than of selected visual or behavioral successes. This leads to our central question: *can coding agents build worlds that are functional and versatile across the activities they are meant to support?*

In this work, we introduce 3DSceneBench to answer this question. For each scene we author a *reference world*, a simulated room whose objects, mechanisms, and physical properties are verified by human so that every test has a controlled outcome. A coding agent receives posed pictures rendered from it and descriptions of the tasks, and it writes, renders, simulates, and revises a world of its own in code before submitting one finished version. Hidden tests then act on the submitted world, from single physical interactions to multistage tasks, and compare each outcome with the outcome of the same test in the reference world. From these comparisons we report appearance ($\mathcal{A}$), geometry ($\mathcal{G}$), and functionality ($\mathcal{F}$), which separate what a world looks like, how accurately it is built, and what it can do. Our contributions are:

- **An executable protocol for whole-world evaluation.** Every test acts on the same submitted world, so an agent scores well only by building one world that serves all the uses of its room, rather than a world that fits a single task.
- **A diverse and demanding benchmark.** 3DSceneBench covers 49 whole-room scenes across seven categories (Fig. 1), from homes and restaurants to laboratories and industrial sites, with 14,219 objects, 471 public tasks, and 36,871 admitted tests.
- **A large-scale evaluation and diagnosis.** Across 1,764 builds from ten frontier models, the highest $\mathcal{F}$ is 42.1 and the highest full-task completion rate is only 28.6%. We discuss what these results imply for evaluating and improving how coding agents build usable 3D worlds.

2. Related work

Building interactive 3D worlds. An interactive 3D world must specify not only what a scene looks like but also how its objects are separated, supported, and articulated, and how they respond to actions. Methods that build such worlds follow two branches. *Scene generation* synthesizes new scenes from text or rules and is judged by plausibility rather than by agreement with a particular scene. Early generators sample layouts procedurally or from learned priors [11, 36, 40, 47]. Later work makes generated scenes simulation-ready by guiding layouts with collision and reachability constraints [61] or by using language models to plan object placement and physical relations [45, 54, 62]. *Scene reconstruction* recovers a specific scene from images, video, or scans, and must therefore agree with the observations. Reconstruction has progressed from recovering object shapes and layout [4, 7, 18] to enforcing stable support and contact [22, 24, 55], recovering joints and movable parts [6, 20, 56], and estimating mass, friction, and other material properties [37, 60, 64]. Across both branches, a growing number of methods use language-model agents that write the world as an executable program and refine it with rendering and simulation feedback [17, 19, 21, 38, 44, 58]. 3DSceneBench evaluates how well general-purpose coding agents build functional and versatile worlds in this way.

Table 1. **3DSceneBench evaluates both the functionality and the versatility of a built world.** Scope is one object (Object), a small scene around one edit, event, or task (Local), or a whole environment (Env). $\mathcal{A}/\mathcal{G}$ marks appearance and geometry evaluation. *Unseen interactions* test responses to actions the inputs never show. *Diverse tasks* evaluate distinct tasks in one world without rebuilding. Circles mark full (●), partial (◑), or no (○) coverage.

Benchmark	Input	Scope	$\mathcal{A}/\mathcal{G}$	Unseen interactions	Diverse tasks
3DCodeBench [12]	Image/Text	Object	●/●	○	○
BlenderGym [16]	Image	Local	●/●	○	○
Code4Scene [63]	Image/Text	Env	●/●	○	○
SceneActBench [67]	Image/Video	Env	○/●	○	○
BVB [49]	Video	Env	●/◑	○	○
4DCodeBench [42]	Video	Local	●/●	○	○
PhysCodeBench [59]	Text	Local	●/○	○	○
PhysCodeGym [41]	Video	Local	●/○	●	○
WorldCoder-Bench [29]	Text	Local	◑/○	◑	○
Video2World [48]	Video	Local	◑/●	◑	○
3DSceneBench	Image/Text	Env	●/●	●	●

Evaluating built worlds. Benchmarks for code-built 3D content differ in the evidence they use to judge the result. *Static correspondence* compares the built artifact with a target without acting on it: 3DCodeBench [12] and BlenderGym [16] evaluate objects and graphics edits, while Code4Scene [63], SceneActBench [67], BVB [49], and LEGO-Bench [26] evaluate scenes by appearance, geometry, task fulfillment, or static physical validity. *Executed behavior* runs the built world and compares what happens with an observed event or a specified behavior. 4DCodeBench [42], VisPhyWorld [28], and PhysCodeBench [59] compare simulated motion with an observed or described event, and PhysCodeGym [41] additionally applies counterfactual interventions to that event. WorldCoder-Bench [29] probes browser-based worlds against hidden behavioral contracts, and Video2World [48]

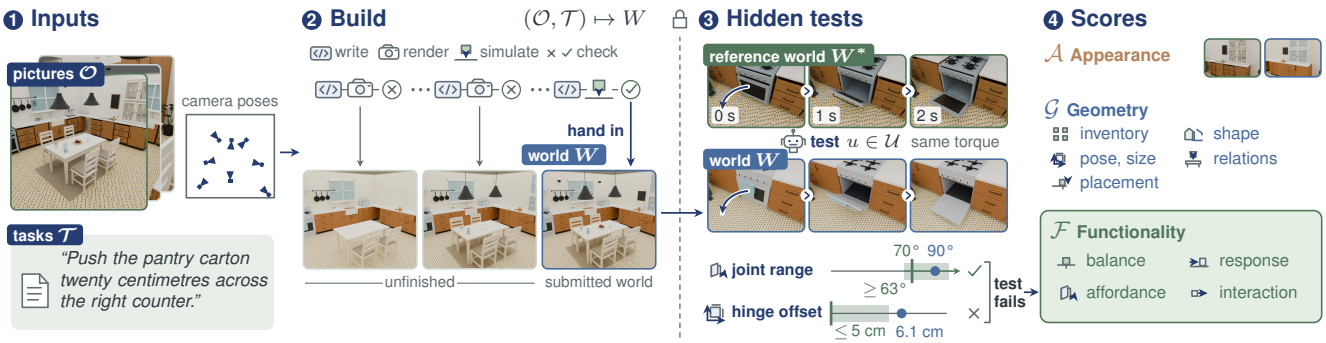


Figure 2. **Evaluation pipeline. (1) Inputs.** A coding agent receives posed pictures $\mathcal{O}$ rendered from the reference world W^* and public tasks $\mathcal{T}$. **(2) Build.** It writes its world W as code, renders, simulates, and checks it repeatedly within a fixed budget, and then submits one world. **(3) Hidden tests.** The grader keeps W^* and all tests $\mathcal{U}$ hidden, from object tests to the checks of every task. A fixed executor runs each test in both W^* and W and compares the outcomes within tolerances set on W^*. **(4) Scores.** The grader reports appearance $\mathcal{A}$, geometry $\mathcal{G}$, and functionality $\mathcal{F}$. Functionality is the primary score used to rank coding agents.

runs a demonstrated task with agent-written controls. In each of these protocols, the instance specifies what the world must reproduce: a target, an observed event, or a prescribed behavior. 3DSceneBench differs in two respects. First, fixed executors run object-level tests and multistage tasks in one submitted world and in a hidden reference world, so the same world is tested on many uses, including interactions that its inputs never show. Second, 3DSceneBench evaluates each world at every level, from appearance and geometry to object behavior and complete multistage tasks. Table 1 summarizes these differences.

3. Building 3DSceneBench

3DSceneBench evaluates whether coding agents can build functional and versatile 3D worlds from visual observations and task instructions. To evaluate these capabilities, we construct a reference world and its tests for each scene (Sec. 3.1), define the protocol under which agents build and submit one world (Sec. 3.2), and evaluate that world for appearance, geometry, and functionality (Sec. 3.3). Figure 2 summarizes the benchmark workflow.

3.1. Reference worlds and tests

Evaluating a world across diverse physical uses requires controlled reference outcomes. Images and videos provide evidence of appearance, layout, and recorded interactions, but do not by themselves specify responses to new actions. We therefore construct a simulated *reference world* W^* for each benchmark instance. It specifies the objects, their geometry and appearance, physical properties, spatial relations, articulation constraints, and initial state, allowing us to inspect the scene and repeat physical tests under controlled conditions.

Authoring the reference world. A loop of three agents builds each reference world under human supervision. A *planner* inspects reference images or panorama views, proposes an inventory of objects and their intended uses, and decides what to build or revise next. A *builder* implements the plan. It retrieves or procedurally constructs assets, assembles them into a room, writes the tasks as executable code, and fixes execution failures. A *critic* checks each result against the written requirements, the reference images, and simulation feedback, including rest stability and articulated motion, and returns its findings to the planner. Each asset is built to be used as well as seen. Movable objects are separate bodies, doors and drawers have joints with travel limits, and containers are hollow so that objects can be placed inside. The loop proceeds through the assets, the assembled room, and the tasks. Each stage advances only after the critic accepts it, and the critic checks the whole room again after the tasks are added, because task-related fixes can break objects that were accepted earlier.

Creating the tests. The authored activities yield public task descriptions $\mathcal{T}$. Their executable checks, together with object-level probes and additional unannounced task instances, form a private test set $\mathcal{U}$. Calibration runs each candidate test on W^* and freezes its action, reference outcome, and acceptance criterion. Because observations do not uniquely determine mass or friction, tests assess behavior within tolerances rather than require exact parameter recovery. Admission keeps a test only if its reference outcome stays within tolerance when object positions are slightly perturbed and when mass and friction vary over plausible ranges. For tasks with custom checks, we also create deliberately broken versions of W^*, and the check must fail on each of them. This shows that the check detects errors rather than passing any world. Before release, submitting W^* itself must score 100 within numerical tolerances, which confirms that the reference world, the tests, and the grader agree.

Trusting the reference. The agent loop performs the construction, but on its own it does not produce a reliable reference world. We therefore steer it throughout construction by writing direct feedback, supplying additional refer-

Table 2. **Main results on 3DSceneBench (L0 setting).** Each value averages three builds of each of the 49 scenes, and rows are sorted by $\mathcal{F}$. $\mathcal{A}$, $\mathcal{G}$, and $\mathcal{F}$ are 0–100 scores computed from the measurements in their group. Full is the share of tasks completed in full and is reported but not scored. Valid is the share of builds that load and pass the simulation gate, and invalid builds score 0. Shading runs from light (worst) to dark (best) within each column. **Bold** and underline mark the best and second-best values, excluding Valid.

		Appearance		Geometry							Functionality					Scores			Per build		
Agent	Effort	SigLIP	DINO	Invent.	Shape	Pose and size			Relat.	Place.	Bal.	Resp.	Afford.	Inter.	Full	$\mathcal{A}$	$\mathcal{G}$	$\mathcal{F}$	Valid	Time	Cost
		cos↑	cos↑	F1%↑	F%↑	%↑	cm↓	ratio↓	%↑	%↑	%↑	%↑	%↑	%↑	%↑	pts↑	pts↑	pts↑	%↑	min↓	$↓
Claude Opus 5.5	high	0.93	0.79	70.4	67.7	39.8	9.7	1.29	51.6	61.2	58.0	40.3	37.0	33.1	26.6	69.5	58.1	**42.1**	100	86	14
Claude Opus 5.5	max	0.93	0.79	70.9	68.4	39.6	9.2	1.29	50.1	59.6	57.6	39.5	37.3	**33.9**	**28.6**	70.5	57.7	42.1	100	123	24
Gemini 4.0 Argon	high	0.92	0.77	**76.0**	68.2	**48.4**	**7.4**	**1.28**	**56.6**	**61.5**	**59.8**	39.4	**38.1**	30.5	24.6	66.7	**62.1**	42.0	100	116	9.9
GPT-6 Astra	max	**0.95**	**0.85**	69.5	**69.3**	41.1	9.9	1.31	54.2	56.3	55.4	**43.2**	35.7	30.1	23.5	**79.3**	58.0	41.1	99	126	50
GPT-6 Astra	high	0.94	0.84	66.7	68.8	37.1	11.3	1.31	52.1	54.0	53.2	39.9	34.0	31.2	24.2	75.6	55.7	39.6	98	96	36
Kimi K3.1 Pro	high	0.90	0.69	47.0	59.7	8.5	27.0	1.43	26.5	33.4	33.0	18.5	16.1	16.1	12.3	55.3	35.0	20.9	100	91	5.5
DeepSeek V4.1 Flash	high	0.88	0.60	43.9	52.4	4.4	41.8	2.02	19.0	25.4	28.4	10.9	10.9	20.4	11.8	45.1	29.1	17.7	100	89	**0.22**
Qwen3.8 Max	high	0.84	0.56	36.5	53.0	3.2	37.7	1.46	15.0	22.8	22.2	12.5	8.7	9.7	6.8	39.0	26.1	13.3	98	107	3.7
MiniMax M3.1 Flash	high	0.88	0.61	38.0	52.7	4.5	32.6	1.53	9.4	21.8	24.3	10.1	10.2	7.2	3.2	48.2	25.2	12.9	100	91	0.95
GLM-5.3 Flash	high	0.80	0.54	36.0	46.5	4.3	34.7	1.51	12.7	20.0	21.7	10.4	7.8	11.0	9.6	36.9	23.9	12.7	91	129	0.99
MiMo V2.6 Pro	high	0.85	0.58	34.9	51.3	2.4	37.6	1.52	12.0	16.6	16.7	7.1	10.7	11.7	8.5	40.0	23.5	11.6	100	111	0.66
Mistral Large 4	high	0.84	0.55	41.2	43.7	5.1	29.3	1.63	10.4	22.8	22.8	8.1	7.6	6.2	2.6	36.2	24.7	11.1	100	**79**	2.7

ence images, and extending the asset libraries or tools when needed. Three human experts then review each room independently, compare their notes, and accept the room only after all findings are resolved. Each reference world thus receives more information, budget, and human guidance than an evaluated agent.

Scene coverage. The resulting benchmark covers 49 scenes across seven categories: homes, hospitality and food service, retail, offices and recreation, healthcare and assisted living, science and research, and industry and infrastructure. The scenes contain 14,219 objects in total, 290 per scene on average, and 471 public tasks. These settings encompass diverse physical interactions: articulated mechanisms support opening and pressing, containers support holding and carrying, and fluids, soft bodies, and cloth introduce pouring and deformation. Rigid behavior is evaluated in MuJoCo [50], with supported non-rigid behavior evaluated in Genesis [5, 13].

3.2. Benchmark protocol

For each scene, every coding agent receives the same public inputs and builds a world W within a common budget.

Input settings. We define two input settings that share the reference world, tests, and grading. The primary setting, L0 (observations only), provides 8 main views of the room and up to 16 detail views of objects and mechanisms that the main views miss, all rendered from W^* with camera poses and intrinsics. It also provides the public tasks and descriptions of the object-test families. The diagnostic setting, L1 (observations plus specifications), additionally provides an object inventory, dimensions, joint travel, depth maps, and room-specific descriptions of what the tests check. Neither setting reveals mass, friction, the reference world, the hidden tasks, or the held-out evaluation cameras. We compare agents in L0 and use L1 only for analysis.

Construction and submission. Agents iteratively edit simulation code and assets using rendering, simulation, and measurement tools. They may compare their renders with the observations and test their own worlds, but cannot query the reference world's responses or obtain private benchmark scores. A public checker reports packaging, loading, and supported structural checks. Every run uses the same tools in an isolated container with no GPU and no network access beyond the model API. A run ends at 180 minutes or 10 million output tokens, including reasoning, whichever is exhausted first. The agent then submits its saved world, a MuJoCo scene file with its assets, for grading. Each scene is packaged as a Harbor task, so a new agent runs a room with one command, and grading uses a private task twin that the agent never sees.

3.3. Evaluation and metrics

Given a submitted world W, the grader compares it with the reference world W^* using held-out views, geometric measurements, and the frozen tests in $\mathcal{U}$. We report scores from three perspectives: appearance ($\mathcal{A}$), geometry ($\mathcal{G}$), and functionality ($\mathcal{F}$), each on a 0–100 scale. Individual measurements accompany each aggregate score.

One world across tests. The grader first matches submitted objects one-to-one to reference objects by category, position, and size. Geometry is measured on the matched pairs, and every test acts on matched objects, so a test whose target is unmatched fails. Each test applies one fixed action to its target objects and compares the outcome with the reference outcome within a tolerance. For example, a push test passes when the force that first moves a cup lies within a factor of two of the force that moves it in W^*. Object tests use scripted forces and motions, and tasks use scripted controllers with privileged object poses, including a simulated mobile manipulator. Because these executors are fixed, a

failed test reflects the world rather than a controller. Each object test starts from the world's initial state, while state carries across the stages of a task, and the same submitted world serves every test without task-specific rebuilding.

Appearance. We render W and W^* in their initial states from held-out cameras under matched lighting. We compare paired renders with SigLIP-2 [51] and DINOv3 [43] cosine similarities, which measure semantic and visual agreement, and with LPIPS [66], which measures perceptual distance. Each measure is averaged over cameras.

Geometry. Geometry has five components, covering which objects exist, their shapes, their poses and sizes, their spatial relations, and how they are placed. **Inventory** measures matched-object precision and recall. **Shape** measures surface F-score after removing position and scale. **Pose and size** measures center distance and the worst side-length ratio, and records whether both lie within tolerance. **Relations** checks required spatial relationships, including support. **Placement** checks whether objects are supported, free of excessive penetration, and inside the room.

Functionality. Functionality comprises four levels. **Balance** checks settling, drift, physical validity, and edge support. **Response** checks reactions to applied actions, such as pushing a cup, tilting a bottle, or pressing a cushion. **Affordance** checks mechanisms, containment, grasping, and carrying contents. **Interaction** checks tasks stage by stage under fixed executors. Because the levels test the same objects in different ways, a world cannot pass one level by breaking another. For example, fixing every object in place fails the balance check that movable objects are free bodies, as well as every push, tilt, and carrying test on those objects.

Scores. Each view is mapped to a 0–100 score. For $\mathcal{A}$, the SigLIP-2 and DINOv3 similarities are rescaled so that the reference world's empty room shell scores 0 and an identical render scores 100, and then averaged. $\mathcal{G}$ averages the scores of its five components, and missing reference objects remain in every denominator. $\mathcal{F}$ averages the scores of its four levels, which use two kinds of tests. Balance, response, and affordance use object tests. Most object tests score 1 if they pass and 0 if they fail, and a few, such as tests on deformable objects, give partial credit. Each of these three levels scores the mean over its object tests. Interaction uses tasks, which are scored by stage. A task counts the stages completed before its first failure, so a four-stage task that fails at its third stage completes two of its four stages. Interaction scores the share of all task stages that are completed. We also report *Full*, the share of tasks completed in full.

A world that cannot load scores zero on all three views. A world that fails the simulation gate, which checks for valid physical parameters and stable simulation, fails all its functionality tests. Other failures follow the rules of each test family.

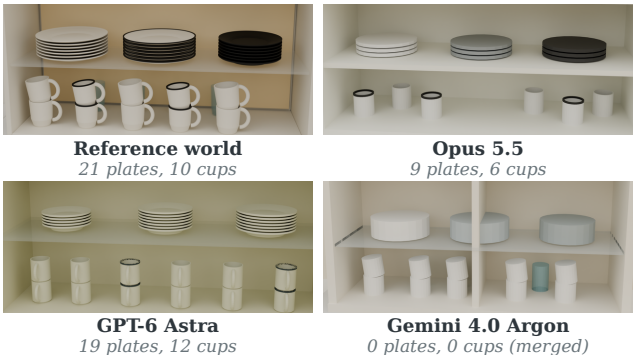


Figure 3. **Agents leave the room incomplete.** Two shelves of the dish hutch in the Home Kitchen, from the same camera, in the reference world and in the worlds of three agents, with the number of plates and cups that each world contains as separate objects.

4. Main results

4.1. Setup

We evaluate ten frontier models in three coding harnesses. Claude Opus 5.5 [3] runs in Claude Code [2], and GPT-6 Astra [35] runs in Codex [34]. The other eight models run in OpenCode [8]: Gemini 4.0 Argon [15], Kimi K3.1 Pro [32], DeepSeek V4.1 Flash [10], Qwen3.8 Max [39], MiniMax M3.1 Flash [30], GLM-5.3 Flash [14], MiMo V2.6 Pro [57], and Mistral Large 4 [31]. All agents run at high reasoning effort, and Opus and Astra also run at max effort, giving 12 configurations. Each configuration builds every scene three times in the L0 setting under the protocol of Sec. 3.2, for 1,764 builds in total. To reduce run-to-run variation, we average the three builds of each scene and then average over the 49 scenes. Instructions and harness details can be seen in Appendix B.

4.2. Appearance and geometry

$\mathcal{A}$ measures how closely a world looks like the reference world, and $\mathcal{G}$ measures how well it reproduces the reference objects and their layout. As seen in Tab. 2, no single agent leads both. Astra at max effort builds the worlds that look most like the reference ($\mathcal{A}$ 79.3), while Gemini has the highest $\mathcal{G}$ (62.1) and leads six of the seven geometry measures.

Opus, Gemini, and Astra lead the seven open-weight models on both scores, and the gap lies more in layout than in look. For example, at high effort, Kimi and Astra reach similar SigLIP-2 similarities (0.90 and 0.94). However, only 8.5% of Kimi's matched objects lie within the pose-and-size tolerance, against 37.1% for Astra, and Kimi's inventory score is 47.0, against 66.7 for Astra.

4.3. Functionality

$\mathcal{F}$ measures whether the objects and mechanisms of a world respond to physical interactions, and whether the same world supports many different uses. It averages four levels that ask progressively more of the world. As seen in Tab. 2,

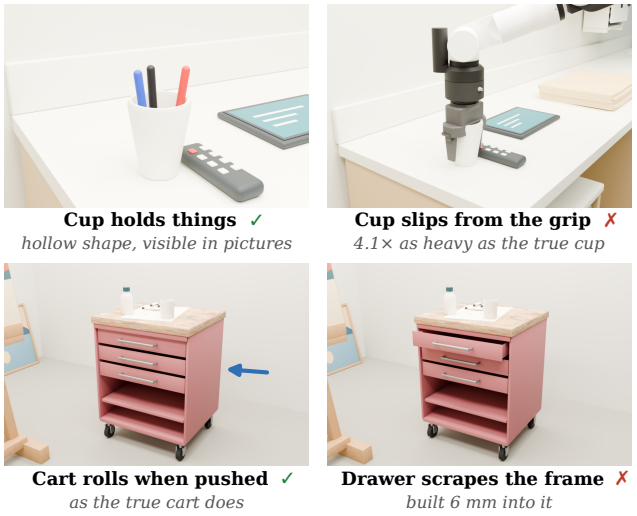


Figure 4. **One object passes one use and fails another.** Illustrations with the reference objects; each outcome is that of the same object in one of Opus's worlds. Its cup holds things but, built 4.1 times as heavy as the reference cup, slips from the grip that lifts the reference one. Its drawer cart rolls like the reference cart, but its drawers are built 6 mm into the frame and scrape through it when pulled. The arm illustrates the grasp test's bounded grip.

balance is the highest level in every configuration, while response, affordance, and interaction remain lower. For Opus at high effort, balance reaches 58.0, while the other three levels lie between 33.1 and 40.3. Objects that rest stably can still fail when pushed, opened, or used in a task, and even the best configuration loses more than half of the functionality points.

Opus, Gemini, and Astra are nearly tied on $\mathcal{F}$. Their best configurations lie within one point of each other (Opus 42.1, Gemini 42.0, Astra 41.1), but their strengths differ by level. Gemini leads balance and affordance, Astra leads response, and Opus leads interaction. The open-weight models fall much further behind. At high effort, for example, Kimi reaches an $\mathcal{F}$ of 20.9, about half of Astra's 39.6.

In addition, a better look does not bring better function. Astra at max effort scores 12.6 points higher than Gemini on $\mathcal{A}$, yet 0.9 points lower on $\mathcal{F}$. Geometry is closer to functionality, because every test acts on matched objects and fails when its target is unmatched. The open-weight models recover fewer reference objects, so many of their tests have no object to act on.

Finally, complete tasks are the hardest. Each stage of a task continues from the state that the previous stage left, so a task succeeds only if the world supports every use in sequence. Even Opus at max effort, the best configuration on tasks, completes only 28.6% of its tasks in full. Its interaction score, the share of task stages completed, is 33.9.

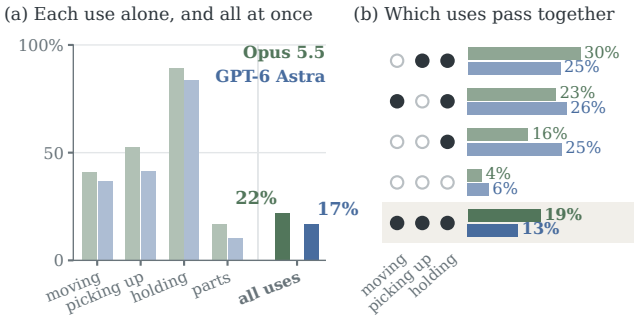


Figure 5. **Few objects serve every use.** (a) Matched objects tested for at least two uses: share that passes each use alone, and every tested use at once. (b) Matched objects tested for moving, picking up, and holding: the most frequent combinations of uses that pass (filled dots) and the share of objects with each. High effort.

4.4. Validity, time, and cost

In Tab. 2, 8 of the 12 configurations deliver a valid world in every build, and the others do so in most builds. The differences in the scores therefore come mainly from the worlds that agents build, not from failed builds.

Cost varies far more than time. Build times range from 79 to 129 minutes, all within the 180-minute budget, while costs at list prices range from $0.22 to $50 per build. Gemini offers the best balance of functionality and cost. It nearly ties for the highest $\mathcal{F}$ at $9.9 per build, less than Opus ($14) and Astra ($36) at high effort. Opus at high effort is the fastest of the leading three, at 86 minutes per build, and DeepSeek is the cheapest configuration, at $0.22 per build. The open-weight models cost much less in general, but they also reach a much lower $\mathcal{F}$.

Spending more on the same model helps little. Opus and Astra also run at max effort, which lets the model reason longer at each step. In Tab. 2, this adds cost and time but little functionality. For Opus, max effort costs 1.7 times as much and takes 37 minutes longer, while $\mathcal{F}$ stays at 42.1. For Astra, it costs 1.4 times as much, takes 30 minutes longer, and raises $\mathcal{F}$ by only 1.5 points. Even at max effort, both agents submit about an hour before the 180-minute budget runs out. These runs suggest that more reasoning effort alone does not close the gap. Section 5 examines what agents build and check before they submit.

5. Discussion

5.1. Why do agents stop early?

We find that agents lose most of their functionality on objects they never build. At high effort, 52% of the $\mathcal{F}$ points that Opus loses and 49% of those that Astra loses come from tests that never run, because the object they need is missing or unmatched. About two thirds of this loss concerns objects that are not built as separate objects. Some are left out entirely, and others are merged into the mesh of the fur-

niture that holds them, so no test can act on them. Objects that exist but lie outside the pose-and-size tolerance account for about another 30% of the lost points, and objects within tolerance that still fail account for about 15%. The missing objects are rarely the ones that the tasks mention. Tests on objects used by the public tasks find no matched object in 7–10% of cases, while tests on the rest of the same room find none in 36–38%. Agents also leave much of their time unused. Opus and Astra submit after 86 and 96 minutes on average, about half of the 180-minute budget, and Gemini after 116 minutes (Tab. 2). Agents thus treat the task list as the room's specification and stop once it is covered. Figure 3 shows a typical case. On two shelves of the kitchen's dish hutch, Opus builds 9 of 21 plates and 6 of 10 cups.

Agents stop early partly because their own checks do not show what is missing or wrong. Agents test their worlds while building, for example by opening a drawer in simulation and checking how far it moves. These checks can only test objects that the agent has built, so they never reveal a missing one. Even for objects that exist, agents compare the result with values they set themselves, such as the travel limit in their own code, rather than with what the pictures show. Such a check passes even when the limit is wrong. None of the joint checks that Opus and Astra run in the builds we inspected takes its expected value from the input pictures or task descriptions. The agent sees its checks pass and concludes that the room is done, while the hidden tests compare the drawer with the one in the reference world.

Although all three leading agents stop early, they build in different ways. Opus builds selectively and checks its world by eye. It creates 0.87 candidate objects per reference object, and 80% of its builds place a render of the world next to the input pictures. Astra builds more broadly and looks back less often, with 1.75 candidates per reference object and such comparisons in 35% of its builds. On the dish hutch, it builds nearly all of the plates and cups (Fig. 3). Gemini recovers the most objects. It has the highest inventory score, 76.0 against 70.4 for Opus and 66.7 for Astra, and places the most matched objects within the pose-and-size tolerance (48.4%). Still, it builds the plates and cups of the hutch into the cabinet itself, so none of them can be tested, and its $\mathcal{F}$ of 42.0 is no higher than Opus's. These habits change which objects match, but none of the three agents completes the room. Future agents should therefore check the observed inventory and mechanisms, not only the task list, before they stop.

5.2. Why is it hard to make one world versatile?

A world supports diverse uses only when its objects can fulfill the different roles those uses require. A cup, for example, must hold its contents and be lifted, and a drawer cart must roll when pushed and let its drawers slide out (Fig. 4). We therefore group the object tests by the use they exercise: moving the object (push, slide, and tilt), picking it up (grasp), holding contents (container and turnover), and operating articulated parts (joints). Standing stably is a prerequisite shared by these uses. Success in one use rarely extends to all the others. Among matched objects tested for at least two uses, only 22% of Opus's objects and 17% of Astra's pass every tested use (Fig. 5a). Within this group, even among objects that pass at least one use, 72% and 76%, respectively, fail another.

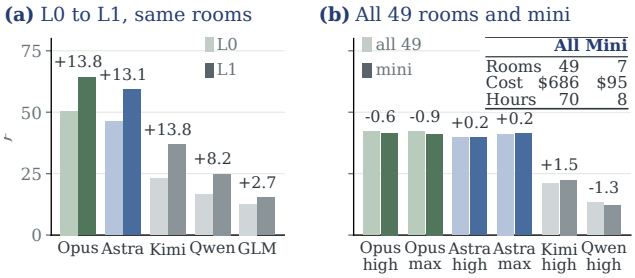


Figure 6. **Scores follow what the agent is told, not the number of rooms.** (a) $\mathcal{F}$ of the same agent on the same rooms in L0 (light) and L1 (dark). Labels give the gain. (b) $\mathcal{F}$ on all 49 rooms (light) and on 3DSceneBench-mini (dark). Labels give the difference. The inset gives rooms, cost, and hours of one Opus build per room.

Some uses are much harder to support than others. Among Opus's objects that pass at least one other use, only 7% fail to hold contents, compared with 59% that fail movement tests, 47% that fail grasp tests, and 91% that fail joint tests. These differences point to distinct requirements: holding contents mainly requires a hollow shape, whereas moving and grasping also depend on mass and friction. Articulated motion further requires appropriate joint axes, travel, clearance, and damping. Visual observations constrain these properties to different degrees: a container's shape is often visible, while mass and friction cannot be observed directly, and joint properties are only partly visible.

Correcting mass and friction confirms that physical parameters account for part of this gap. Replacing only these parameters with their reference values, while keeping shape, pose, and joints unchanged, improves $\mathcal{F}$ in all six tested worlds, from 45.3 to 51.7 on average. Push pass rates rise from 67% to 86%, and grasp pass rates from 44% to 84%. Joint pass rates remain near 10%, showing that these corrections leave a separate articulation problem unresolved. During construction, agents test worlds using the physical parameters they have assigned themselves (Sec. 5.1), so passing these checks does not show that the parameters are correct. Building a versatile world therefore requires checking each object across its different uses and identifying which physical properties each use depends on.

5.3. Do agents lack information about the room?

How can coding agents build better worlds, and do they simply lack information? We compare two input settings



Figure 7. **One room built by seven methods.** The Pump Repair Workshop from the same camera in the reference world, in the worlds of three coding agents (top), and in the worlds of four scene methods (bottom). Under each picture are the method's input and its $\mathcal{A}$, $\mathcal{G}$, and $\mathcal{F}$ on this room (one run each). The four scene methods run on GPT-6 Astra as their language model.

(Sec. 3.2). L0 gives posed pictures and public task descriptions. L1 also gives the inventory, object dimensions, joint travel, depth, and room-specific test descriptions. It acts as an oracle setting for these specifications and serves only for analysis. On the same rooms, L1 raises Opus's $\mathcal{F}$ from 50.5 to 64.3 and Astra's from 46.2 to 59.3, and every agent in Fig. 6a gains 2.7 to 13.8 points. Missing information thus explains part of the loss, yet even with it no agent comes close to a full score. More views alone do not help. With a walkthrough video replacing still pictures, Opus's $\mathcal{F}$ on five rooms is 44.5, versus 43.9 with pictures. Better agents therefore need to infer more from the pictures they have, not to see more of the room.

Developing such improvements requires repeated evaluation, which is costly. One Opus build per room at high effort costs $686 on the full benchmark. We therefore release 3DSceneBench-mini as a validation set of seven full rooms, one per category. Its scores closely track the full benchmark, with $\mathcal{F}$ differing by at most 1.5 points across the six configurations in Fig. 6b, while the same Opus evaluation costs $95. Researchers can use mini to iterate on algorithms and coding harnesses, then report final results on the full benchmark.

5.4. Can other 3D scene methods be evaluated?

The benchmark is not specific to coding agents. It grades the submitted world, not the method that produced it, so any method that outputs a 3D scene can take the same hidden tests. We run four scene methods on one room, each with its native input rendered from the same reference world: a video for AHA-3D [1], RGB-D scans for LiteReality-Agent [19], a single picture for HARMONY [46], and a caption for SceneSmith [38]. All four use GPT-6 Astra as their language model. Figure 7 shows their worlds below those of three coding agents on the same room. The best scene methods match the coding agents in appearance but not in function. AHA-3D and LiteReality-Agent reach an $\mathcal{A}$ of 72.7 and 77.5, inside the range of the coding agents (56.8 to 79.4), yet an $\mathcal{F}$ of only 4.6 and 4.8. No scene method exceeds an $\mathcal{F}$ of 5.1, whereas the coding agents reach 56.3 to 61.4. The functional tests also indicate what each output lacks. AHA-3D exports doors and drawers without joints. LiteReality-Agent exports bodies, collisions, and joints, but matches only 16 of the room's 53 objects and supplies two drawer joints for the whole room. HARMONY returns 5 of the 53 objects. SceneSmith matches 11, and none of them lies within the pose-and-size tolerance. Scene methods should therefore treat mechanisms and physical values as part of their output rather than as a conversion step after it. This matters most when such worlds are used for robots. A robot can be trained or evaluated in a built world only if it behaves like its room [9, 25, 52, 53, 65], and $\mathcal{F}$ measures this necessary condition.

6. Conclusion and future work

We present 3DSceneBench, a benchmark that evaluates whether coding agents can build functional and versatile 3D worlds from visual observations of whole rooms. Across ten models in 12 configurations, the highest functionality score is only 42.1, and agents lose most of these points on objects they never build. The challenge is to make object geometry, articulation, and physical properties jointly support different uses in one construction.

Several directions follow from these results. Since agents already leave much of a single room unbuilt, larger environments such as multi-room homes and whole facilities would test whether they can keep track of a full inventory at that scale. Closer physics would sharpen the measurement itself, as our reference worlds are simulated and mass and friction decide much of how objects respond, so measured physical properties and evaluation across several simulators would bring the tests nearer to real behavior. Dynamics is a natural next step. Videos of a room in use constrain how its objects behave over time [42, 48], so adding such observations and tests that run over time would extend the benchmark from static rooms to scenes that change. A harness in which an agent can request video evidence of a mechanism it is unsure about, and revise its world in response, would bring that world closer to the physics of the room. Robots are the other direction. Each reference world includes a simulated robot and scripted tasks, so 3DSceneBench can serve as a playground for robot learning and for physical recursive self-improvement [27]. Our scores, however, measure correspondence to a simulated reference under fixed executors, so transfer to real rooms and to learned policies remains to be validated.

References

[1] AHa-3D contributors. AHa-3D: Agentic Tool Use for Real2Sim with GPT-6 Astra. https://github.com/KevinXu02/aha-3d, 2026. Repository, revision 82f4b11. 8

[2] Anthropic. Claude Code. https://github.com/anthropics/claude-code/releases/tag/v2.1.309, 2026. Version 2.1.309. 5

[3] Anthropic. Claude Opus 5.5 system card. https://www.anthropic.com/claude-opus-5-5-system-card, 2026. 5

[4] Andreea Ardelean, Mert Özer, and Bernhard Egger. Generalizable 3d scene reconstruction via divide and conquer from a single view. In *International Conference on 3D Vision (3DV)*, 2025. 2

[5] Genesis Authors. Genesis: A generative and universal physics engine for robotics and beyond. https://github.com/Genesis-Embodied-AI/genesis-world, 2024. 4

[6] Qiuyu Chen, Aaron Walsman, Marius Memmel, Kaichun Mo, Alex Fang, Dieter Fox, and Abhishek Gupta. URDFormer: A Pipeline for Constructing Articulated Simulation Environments from Real-World Images. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024. 2

[7] Xingyu Chen, FU-JEN CHU, Pierre Gleize, Kevin J Liang, Alexander Sax, Hao Tang, Weiyao Wang, Michelle Guo, Thibaut Hardin, Xiang Li, Aohan Lin, Jia-Wei Liu, Ziqi Ma, Anushka Sagar, Bowen Song, Xiaodong Wang, Jianing Yang, Bowen Zhang, Piotr Dollár, Georgia Gkioxari, Matt Feiszli, and Jitendra Malik. Sam 3d: 3dfy anything in images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7220–7232, 2026. 2

[8] OpenCode contributors. OpenCode. https://github.com/anomalyco/opencode/releases/tag/v1.19.3, 2026. Version 1.19.3. 5

[9] Tianyuan Dai, Josiah Wong, Yunfan Jiang, Chen Wang, Cem Gokmen, Ruohan Zhang, Jiajun Wu, and Li Fei-Fei. Automated creation of digital cousins for robust policy learning. In *Proceedings of The 8th Conference on Robot Learning*, pages 4912–4943. PMLR, 2025. 8

[10] DeepSeek-AI. DeepSeek-V4.1-Flash: Pushing the limits of KV cache compression. https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash/blob/main/DeepSeek_V41_Tech_Report.pdf, 2026. 5

[11] Matt Deitke, Eli VanderBilt, Alvaro Herrasti, Luca Weihs, Kiana Ehsani, Jordi Salvador, Winson Han, Eric Kolve, Aniruddha Kembhavi, and Roozbeh Mottaghi. Procthor: Large-scale embodied ai using procedural generation. In *Advances in Neural Information Processing Systems*, pages 5982–5994. Curran Associates, Inc., 2022. 2

[12] Yipeng Gao, Lei Shu, Genzhi Ye, Xi Xiong, Ameesh Makadia, Meiqi Guo, Laurent Itti, and Jindong Chen. 3DCodeBench: Benchmarking Agentic Procedural 3D Modeling Via Code. *arXiv preprint arXiv:2606.01057*, 2026. 1, 2

[13] Genesis AI Team. The role of simulation in scalable robotics, Genesis World 1.0, and the path forward. *Genesis AI Blog*, 2026. 4

[14] GLM-5-Team. GLM-5: from Vibe Coding to Agentic Engineering. *arXiv preprint arXiv:2602.15763*, 2026. 5

[15] Google DeepMind. Gemini 4.0 Argon model card. https://deepmind.google/models/model-cards/gemini-4-argon/, 2026. 5

[16] Yunqi Gu, Ian Huang, Jihyeon Je, Guandao Yang, and Leonidas Guibas. Blendergym: Benchmarking foundational model systems for graphics editing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18574–18583, 2025. 1, 2

[17] Ziniu Hu, Ahmet Iscen, Aashi Jain, Thomas Kipf, Yisong Yue, David A Ross, Cordelia Schmid, and Alireza Fathi. SceneCraft: An LLM agent for synthesizing 3D scenes as blender code. In *Proceedings of the 41st International Conference on Machine Learning*, pages 19252–19282. PMLR, 2024. 2

[18] Zehuan Huang, Yuan-Chen Guo, Xingqiao An, Yunhan Yang, Yangguang Li, Zi-Xin Zou, Ding Liang, Xihui Liu, Yan-Pei Cao, and Lu Sheng. Midi: Multi-instance diffusion for single image to 3d scene generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 23646–23657, 2025. 2

[19] Zhening Huang, Yueyan Li, Johnathan Chiu, Xiaoyang Lyu, Matt Zhou, Yuxin Yao, Joan Lasenby, and Shangzhe Wu. LiteReality-Agent: An Agentic System for Interactable 3D Indoor Scene Reconstruction. *arXiv preprint arXiv:2610.01863*, 2026. 1, 2, 8

[20] Zhao Huang, Boyang Sun, Alexandros Delitzas, Jiaqi Chen, and Marc Pollefeys. React3d: Recovering articulations for interactive physical 3d scenes. *IEEE Robotics and Automation Letters*, 11(5):5954–5961, 2026. 2

[21] Long Le, Jason Xie, William Liang, Hung-Ju Wang, Yue Yang, Yecheng Jason Ma, Kyle Vedder, Arjun Krishna, Dinesh Jayaraman, and ERIC EATON. Articulate-anything: Automatic modeling of articulated objects via a vision-language foundation model. In *International Conference on Learning Representations*, pages 17578–17602, 2025. 2

[22] Inhee Lee, Sangwon Baik, Sungjoo Kim, Hyeonwoo Kim, Hyunsoo Cha, and Hanbyul Joo. SimuScene: Simulation-Ready Compositional 3D Scene Reconstruction from a Single Image. *arXiv preprint arXiv:2606.03994*, 2026. 2

[23] Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabrael Levine, Wensi Ai, Benjamin Martinez, Hang Yin, Michael Lingelbach, Minjune Hwang, Ayano Hiranaka, Sujay Garlanka, Arman Aydin, Sharon Lee, Jiankai Sun, Mona Anvari, Manasi Sharma, Dhruva Bansal, Samuel Hunter, Kyu-Young Kim, Alan Lou, Caleb R. Matthews, Ivan Villa-Renteria, Jerry Huayang Tang, Claire Tang, Fei Xia, Yunzhu Li, Silvio Savarese, Hyowon Gweon, C. Karen Liu, Jiajun Wu, and Li Fei-Fei. Behavior-1k: A human-centered, embodied ai benchmark with 1,000 everyday activities and realistic simulation. *arXiv preprint arXiv:2403.09227*, 2024. 1

[24] Haodong Li, Lulu Shao, Haolin Lu, Yu Fu, Yen-Ru Chen, Seemandhar Jain, and Manmohan Chandraker. φ-Scene: Physically Grounded Image-to-3D Scene Reconstruction. *arXiv preprint arXiv:2606.21596*, 2026. 2

[25] Xuanlin Li, Kyle Hsu, Jiayuan Gu, Oier Mees, Karl Pertsch, Homer Rich Walke, Chuyuan Fu, Ishikaa Lunawat, Isabel Sieh, Sean Kirmani, Sergey Levine, Jiajun Wu, Chelsea Finn, Hao Su, Quan Vuong, and Ted Xiao. Evaluating real-world robot manipulation policies in simulation. In *Proceedings of The 8th Conference on Robot Learning*, pages 3705–3728. PMLR, 2025. 8

[26] Xirui Li, Peng Shi, Mingwen Dong, Sheng Zhang, Zhuoyan Xu, Dongkyu Lee, Shuaichen Chang, Yi Xiang, Lin Pan, and Jiarong Jiang. LEGO-Anything: Coding Agents for 3D Scene Reconstruction. *arXiv preprint arXiv:2609.36380*, 2026. 2

[27] Zhiqi Li, Yuxuan Liao, and Bo Zhu. Recursive Code World Models: Building Complex Worlds through Recursive Scene Programs. *arXiv preprint arXiv:2609.11499*, 2026. 8

[28] Jiarong Liang, Max Ku, Ka-Hei Hui, Ping Nie, and Wenhu Chen. VisPhyWorld: Probing Physical Reasoning via Code-Driven Video Reconstruction. *arXiv preprint arXiv:2602.13294*, 2026. 2

[29] Shuo Lu, Yinuo Xu, Kecheng Yu, Siru Jiang, Yongcan Yu, Yubin Wang, Haitao Yang, Yuxiang Zhang, Bin Wang, Ran He, and Jian Liang. Worldcoder-bench: Benchmarking physically grounded 3d world synthesis. *arXiv preprint arXiv:2606.01869*, 2026. 2

[30] MiniMax. MiniMax-M3.1-Flash. https://huggingface.co/MiniMaxAI/MiniMax-M3.1-Flash, 2026. 5

[31] Mistral AI. Mistral Large 4. https://huggingface.co/mistralai/Mistral-Large-4-1T-A52B/blob/main/assets/technical_report.pdf, 2026. 5

[32] Moonshot AI. Kimi K3.1 Pro. https://www.kimi.ai/blog/kimi-k3-1-pro, 2026. 5

[33] Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. RoboCasa: Large-Scale Simulation of Household Tasks for Generalist Robots. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024. 1

[34] OpenAI. Codex CLI. https://github.com/openai/codex/releases/tag/rust-v0.164.0, 2026. Version 0.164.0. 5

[35] OpenAI. GPT-6 Astra system card. https://deploymentsafety.openai.com/gpt-6-astra, 2026. 5

[36] Despoina Paschalidou, Amlan Kar, Maria Shugrina, Karsten Kreis, Andreas Geiger, and Sanja Fidler. Atiss: Autoregressive transformers for indoor scene synthesis. In *Advances in Neural Information Processing Systems*, pages 12013–12026. Curran Associates, Inc., 2021. 2

[37] Sneha Paul, Guile Wu, Bingbing Liu, and Dongfeng Bai. PhysVGGT: Feed-Forward Dense Physical Property Estimation from A Single Image. *arXiv preprint arXiv:2609.18920*, 2026. 2

[38] Nicholas Pfaff, Thomas Cohn, Sergey Zakharov, Rick Cory, and Russ Tedrake. SceneSmith: Agentic generation of simulation-ready indoor scenes. In *Proceedings of the 43rd International Conference on Machine Learning*, pages 98499–98567. PMLR, 2026. 1, 2, 8

[39] Qwen Team. Qwen3.8-Max: A new bar for coding and cowork. https://qwen.ai/blog?id=qwen3.8, 2026. 5

[40] Alexander Raistrick, Lingjie Mei, Karhan Kayan, David Yan, Yiming Zuo, Beining Han, Hongyu Wen, Meenal Parakh, Stamatis Alexandropoulos, Lahav Lipson, Zeyu Ma, and Jia Deng. Infinigen indoors: Photorealistic indoor scenes using procedural generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 21783–21794, 2024. 2

[41] Shuai Shao, Jinchen He, Cunxin Fan, Zihan Guo, Shijian Wang, Qihan Ren, Kangning Zhang, Zunnan Xu, Haoyuan Li, Hao Wang, Jianghao Lin, Weiwen Liu, and Weinan Zhang. PhysCodeGym: Benchmarking Coding Agents for Physically Verifiable 4D Reconstruction. *arXiv preprint arXiv:2610.17288*, 2026. 1, 2

[42] Ruihong Shen, Žiga Kovačič, Peter Kulits, Xingrui Wang, Zizhang Li, Joshua B. Tenenbaum, Alan Yuille, Jieneng Chen, and Jiajun Wu. 4DCodeBench: Benchmarking agents on inverse graphics of dynamic scenes. *arXiv preprint arXiv:2610.03715*, 2026. 1, 2, 8

[43] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seung Eun Yi, Michael Ramamonjisoa, Francisco Massa, Daniel Haziza, Luca Wehrstedt, Jianyuan Wang, Timothée Darcet, Théo Moutakanni, Leonel Sentana, Claire Roberts, Andrea Vedaldi, Jamie Tolan, John Brandt, Camille Couprie, Julien Mairal, Herve Jegou, Patrick Labatut, and Piotr Bojanowski. DINOv3. *Transactions on Machine Learning Research*, 2026. 5

[44] Eachen Soong, Zijian Niu, Haoran Xu, Yihong Dong, Yunbo Wang, and Xiaokang Yang. NeoWorld-3: From Pixels to Code-Defined Physical Worlds. *arXiv preprint arXiv:2610.27690*, 2026. 1, 2

[45] Fan-Yun Sun, Weiyu Liu, Siyi Gu, Dylan Lim, Goutam Bhat, Federico Tombari, Manling Li, Nick Haber, and Jiajun Wu. Layoutvlm: Differentiable optimization of 3d layout via vision-language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 29469–29478, 2025. 2

[46] Shufan Sun, Chen Wang, Enxin Song, Jiatao Gu, and Lingjie Liu. HARMONY: Hierarchical Agentic Reasoning for MONocular Image-to-Scene Synthesis. *arXiv preprint arXiv:2609.26793*, 2026. 8

[47] Jiapeng Tang, Yinyu Nie, Lev Markhasin, Angela Dai, Justus Thies, and Matthias Nießner. Diffuscene: Denoising diffusion models for generative indoor scene synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 20507–20518, 2024. 2

[48] Jinzhou Tang, Zijun Zhang, Jing Yang, Yuchen Yan, Kun Zhou, Lingjun Mao, Ruobing Han, Jinglin Cao, Wenpeng

Xu, Lukun He, Minghao Fu, Fan Feng, and Biwei Huang. Video2world: Benchmarking coding agents for interactive world modeling from embodied videos. *arXiv preprint arXiv:2610.04432*, 2026. 1, 2, 8

[49] Yolo Y. Tang, Daiki Shimada, Jiayue Meng, Jing Bi, Pinxin Liu, Yicheng Wang, Yunzhong Xiao, Zhangyun Tan, Zeliang Zhang, Chao Huang, Susan Liang, Qianxiang Shen, Luchuan Song, Ali Vosoughi, Mingqian Feng, Melika Filvantorkaman, and Chenliang Xu. BVB: Benchmarking Agentic Video Understanding via Programmatic Reconstruction in Blender. *arXiv preprint arXiv:2609.15478*, 2026. 2

[50] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033, 2012. 4

[51] Michael Tschannen, Alexey Gritsenko, Xiao Wang, Muhammad Ferjad Naeem, Ibrahim Alabdulmohsin, Nikhil Parthasarathy, Talfan Evans, Lucas Beyer, Ye Xia, Basil Mustafa, Olivier Hénaff, Jeremiah Harmsen, Andreas Steiner, and Xiaohua Zhai. SigLIP 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features. *arXiv preprint arXiv:2502.14786*, 2025. 5

[52] Marcel Torne Villasevil, Anthony Simeonov, Zechu Li, April Chan, Tao Chen, Abhishek Gupta, and Pulkit Agrawal. Reconciling Reality through Simulation: A Real-To-Sim-to-Real Approach for Robust Manipulation. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024. 8

[53] Xinyi Wang, Heng Hao, Wenjun Hu, Anna Enyu Li, Dizhi Ma, Karthik Ramani, Hankyu Moon, and Yeong-Dae Kwon. ReVeal: A Reconstruction-Aware Real-to-Sim Framework for VLA Policy Evaluation. *arXiv preprint arXiv:2609.23910*, 2026. 8

[54] Yian Wang, Han Yang, Minghao Guo, Xiaowen Qiu, Johnson (Tsun-Hsuan) Wang, Wojciech Matusik, Joshua B Tenenbaum, and Chuang Gan. Physcensis: Physics-augmented llm agents for complex physical scene arrangement. In *International Conference on Learning Representations*, pages 84252–84279, 2026. 2

[55] Hongchi Xia, Chih-Hao Lin, Hao-Yu Hsu, Quentin Leboutet, Katelyn Gao, Michael Paulitsch, Benjamin Ummenhofer, and Shenlong Wang. Holoscene: Simulation-ready interactive 3d worlds from a single video. In *Advances in Neural Information Processing Systems*, pages 32501–32524. Curran Associates, Inc., 2025. 2

[56] Hongchi Xia, Entong Su, Marius Memmel, Arhan Jain, Raymond Yu, Numfor Mbiziwo-Tiapo, Ali Farhadi, Abhishek Gupta, Shenlong Wang, and Wei-Chiu Ma. Drawer: Digital reconstruction and articulation with environment realism. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 21771–21782, 2025. 2

[57] Xiaomi LLM-Core Team. Mimo-v2.6: Scaling reinforcement learning towards self-improvement. *arXiv preprint arXiv:2610.11959*, 2026. 5

[58] Shuzhao Xie, Lelin Wang, Guying Lin, Zhi Wang, and Minchen Li. CoDimRecon: Agentic Reconstruction of Sim-Ready 3D Scenes with Deformable Curves, Surfaces, and Volumes. *arXiv preprint arXiv:2609.36024*, 2026. 2

[59] Tianyidan Xie, Peiyu Wang, Hu Jiaxin, Yuyi Qian, Yuxuan Wang, Shenyi Wang, Rui Ma, Yanlun Peng, Lanjun Wang, Ying Tai, Jian Yang, and Zili Yi. PhysCodeBench: Benchmarking Physics-Aware Symbolic Simulation of 3D Scenes via Self-Corrective Multi-Agent Refinement. *arXiv preprint arXiv:2604.23580*, 2026. 1, 2

[60] Xinli Xu, Wenhang Ge, Dicong Qiu, ZhiFei Chen, Dongyu Yan, Zhuoyun Liu, Haoyu Zhao, Hanfeng Zhao, Shunsi Zhang, Junwei Liang, and Ying-Cong Chen. Gaussianproperty: Integrating physical properties to 3d gaussians with lmms. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 7231–7240, 2025. 2

[61] Yandan Yang, Baoxiong Jia, Peiyuan Zhi, and Siyuan Huang. Physcene: Physically interactable 3d scene synthesis for embodied ai. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16262–16272, 2024. 2

[62] Yue Yang, Fan-Yun Sun, Luca Weihs, Eli VanderBilt, Alvaro Herrasti, Winson Han, Jiajun Wu, Nick Haber, Ranjay Krishna, Lingjie Liu, Chris Callison-Burch, Mark Yatskar, Aniruddha Kembhavi, and Christopher Clark. Holodeck: Language guided generation of 3d embodied ai environments. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16227–16237, 2024. 2

[63] Xiaokang Ye, Siddhant Hitesh Mantri, Zimeng Chen, Edward Zhang, Zhaoxu Zheng, Yuanheng Li, Yizhao Chen, Tianyang Huang, and Lianhui Qin. Code4Scene: Benchmarking Coding Agents for Constructing and Editing 3D Scenes. *arXiv preprint arXiv:2609.36777*, 2026. 1, 2

[64] Albert J. Zhai, Yuan Shen, Emily Y. Chen, Gloria X. Wang, Xinlei Wang, Sheng Wang, Kaiyu Guan, and Shenlong Wang. Physical property understanding from language-embedded feature fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 28296–28305, 2024. 2

[65] Jiaqi Zhang, Feng Ye, Mingjia Yang, Zhihong Chen, Mingkang Xiang, Xinglin Yao, Yanbin Li, Siwei Ma, and Chuanmin Jia. RoboTwin-Phys: Do WAMs and VLAs Understand the Physical World? *arXiv preprint arXiv:2609.26292*, 2026. 8

[66] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. 5

[67] Yifei Zhao, Xiangxin Zhou, Wenhao Yang, Jiaqi Tang, Pu Jian, Huanjin Yao, Jiarui Yao, Haowei Lin, Chunchao Guo, Zhuo Chen, Wenkai Lyu, Jianzhu Ma, Xueqian Wang, and Wenxi Zhu. SceneActBench: Can Agents Act on the 3D Scenes They See? *arXiv preprint arXiv:2607.22393*, 2026. 2